



ΣΥΣΤΗΜΑΤΑ ΕΥΦΥΩΝ ΠΡΑΚΤΩΡΩΝ

1^η εργασία

ΟΝΟΜΑΤΕΠΩΝΥΜΟ	ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ	EMAIL	ΗΜΕΡΟΜΗΝΙΑ
ELVIS AVDULI	E22001	avdulielvis@gmail.com	12/12/2025
ΜΑΡΙΟΣ ΚΑΛΟΥΣ	E22066	kalousmarios3@gmail.com	

7^ο Εξάμηνο



Πίνακας περιεχομένων

1. Ανάλυση της ασκήσης	3
1.1. Περιγραφή και Χαρακτηριστικά Περιβάλλοντος	3
1.2. Ορισμός Χώρου Καταστάσεων (State Space)	4
1.3. Ενέργειες και Μεταβάσεις.....	5
1.4. Συνάρτηση Χρησιμότητας και Κόστους	6
1.5. Μέτρο Απόδοσης (Performance Measure)	7
2. Υλοποίηση του πράκτορα	8
2.1. Αρχιτεκτονική Συστήματος.....	8
2.1.1. GridEnvironment.java.....	8
2.1.2. practical_reasoning_agent.asl	8
2.1.3. Βοηθητικές Κλάσεις.....	8
2.2. Κύριος Βρόχος Ελέγχου	9
2.2.1. Ψευδοκώδικας.....	9
2.2.2. Υλοποίηση σε AgentSpeak	9
2.2.3. Ροή Εκτέλεσης	10
2.3. Λογική Επιλογής Στόχου	10
2.3.1. Αλγόριθμος Υπολογισμού Χρησιμότητας	10
2.3.2. Εκτίμηση Κόστους Συλλογής	11
2.3.3. Αλγόριθμος Επιλογής Βέλτιστου Στόχου.....	11
2.3.4. Υλοποίηση σε Java.....	12
2.3.5. Γιατί Αυτή η Λογική Αυξάνει τη Χρησιμότητα	13
2.4. Λογική Σχεδιασμού Δράσης	13
2.4.1. Αλγόριθμος A*	13
2.4.2. Ευριστική Συνάρτηση	14
2.4.3. Υλοποίηση σε Java.....	14
2.4.4. Ολοκληρωμένη Διαδικασία Σχεδιασμού.....	16
2.4.5. Γιατί Αυτός ο Σχεδιασμός Αυξάνει τη Χρησιμότητα.....	16
2.5. Διαχείριση Δυναμικότητας	17
2.5.1. Μηχανισμός Ανίχνευσης Αλλαγών.....	17



2.5.2. Αντίδραση Πράκτορα	17
2.5.3. Στρατηγική Προσαρμογής.....	17
2.6. Διαχείριση Αποθέματος (Inventory)	18
2.6.1. Στρατηγική "Smart Cleaning"	18
2.6.2. Λογική Απόρριψης.....	18
2.7. Σύνοψη Σχεδιαστικών Επιλογών	19
3. Πειραματική Ανάλυση	19
3.1. Μεθοδολογία Πειραμάτων	19
3.1.1. Πειραματικές Ρυθμίσεις	19
3.1.2. Μετρικές Αξιολόγησης	20
3.1.3. Κλάση ExperimentRunner	20
3.2. Αποτελέσματα Πειραμάτων.....	20
3.2.1. Συγκριτικά Αποτελέσματα: Στατικό vs Δυναμικό	20
3.2.2. Εύρος Τιμών.....	21
3.2.3. Ανάλυση Αποτελεσμάτων	21
3.2.4. Στατιστική Σύγκριση	23
3.3. Ανάλυση Συμπεριφοράς Πράκτορα	24
3.3.1. Θεωρητικό Βέλτιστο vs Πραγματική Απόδοση	24
3.3.2. Ανάλυση Κόστους Μεταφοράς.....	25
3.3.3. Απόδοση Πλοήγησης (A^*).....	25
3.4. Επίδραση Δυναμικότητας	26
3.4.1. Ποσοτική Σύγκριση.....	26
3.4.2. Αίτια Καταστροφικής Απόδοσης	26
3.4.3. Επιτυχημένα Runs στο Δυναμικό	27
3.5. Περιπτώσεις Αποτυχίας και Όρια	28
3.5.1. Παρατηρημένες Αποτυχίες	28
3.5.2. Ανάλυση Αποτυχημένων Runs.....	28
3.5.3. Θεωρητικοί Περιορισμοί	29
3.5.4. Όρια Υλοποίησης.....	29

1. Ανάλυση της ασκήσης

Σκοπός του πράκτορα είναι αρχικά να βάψει μια καρέκλα και ένα τραπέζι χρησιμοποιώντας ένα πινέλο και ένα χρώμα και στη συνέχεια να ανοίξει μια πόρτα χρησιμοποιώντας πάλι δύο αντικείμενα τον κωδικό και το κλειδί. Για να γίνουν αυτά πρέπει ο πράκτορας να περάσει κάποια εμπόδια στο πλέγμα θέσεων του. Τα αντικείμενα είναι είτε στατικά είτε δυναμικά στην διάρκεια ενός επεισοδίου ανά 3 κινήσεις του πράκτορα και οι θέσεις τους διαφέρουν σε κάθε επεισόδιο. Η εργασία αποτελείται από τρία βασικά μέρη:

1. Θεωρητική ανάλυση του πράκτορα
2. Υλοποίηση του πράκτορα σε Jason/Java
3. Πειραματική ανάλυση και ανάλυση των εκτελέσεων

Ο πράκτορας ακολουθεί το μοντέλο BDI δηλαδή:

- **Beliefs** (πεποιθήσεις) : Γνώσεις του πράκτορα για τον κόσμο (π.χ. οι θέσεις των αντικειμένων)
- **Desires**(επιθυμίες): Οι καταστάσεις που θέλει να βρεθεί και να επιτύχει ο πράκτορας
- **Intentions**(προθέσεις): Τα σχέδια που εκτελεί για να φτάσει στις επιθυμητές καταστάσεις. Για αυτήν την υλοποίηση είναι υπεύθυνο το `asl` αρχείο

1.1. Περιγραφή και Χαρακτηριστικά Περιβάλλοντος

Το περιβάλλον του πράκτορα είναι ένα πλέγμα (grid) διαστάσεων 5 *5. Το περιβάλλον, δημιουργείται μέσα στην κλάση `GridEnvironment.java` και ορίζεται ως εξής:

- **Διαστάσεις:** Πλέγμα 5*5 με συντεταγμένες $x, y \in \{1, \dots, 5\}$.
- **Εμπόδια (Obstacles):** Υπάρχουν 4 σταθερά εμπόδια στις θέσεις (2,1), (2,2), (4,4), (4,5), τα οποία μειώνουν τα ελεύθερα κελιά κίνησης.
- **Αρχική Θέση:** Ο πράκτορας εκκινεί πάντα από τη θέση (1,1).
- **Αντικείμενα (Items):** Υπάρχουν 4 αντικείμενα με σταθερές αρχικές θέσεις:
 - ❖ Brush (Πινέλο) στο (1,5)
 - ❖ Key (Κλειδί) στο (1,4)
 - ❖ Color (Χρώμα) στο (5,5)
 - ❖ Code (Κωδικός) στο (3,5)



- **Στόχοι (Goals):** Οι στόχοι είναι τρεις: Τραπέζι (Table), Καρέκλα (Chair), Πόρτα (Door).
 - ❖ Στο στατικό σενάριο, οι θέσεις τους αλλάζουν *μεταξύ* των επεισοδίων αλλά παραμένουν σταθερές *κατά τη διάρκεια* του επεισοδίου.
 - ❖ Στο δυναμικό σενάριο (bonus), οι θέσεις τους αλλάζουν τυχαία κάθε 3 χρονικά βήματα (cycles).

1.2. Ορισμός Χώρου Καταστάσεων (State Space)

Ο χώρος καταστάσεων S του προβλήματος συντίθεται από το καρτεσιανό γινόμενο των επιμέρους υπο-χώρων που περιγράφουν τη γνώση του πράκτορα. Μια κατάσταση $s \in S$ ορίζεται ως το διάνυσμα:

$$s = (P_{ag}, I_{ag}, P_{targets}, S_{targets})$$

Όπου:

1. Θέση Πράκτορα (P_{ag}):

Ο πράκτορας μπορεί να βρίσκεται σε οποιοδήποτε κελί δεν είναι εμπόδιο.

- ο Συνολικά κελιά: 25.
- ο Εμπόδια: 4.
- ο **Δυνατές Θέσεις:** $25 - 4 = 21$.

2. Κατάσταση Αποθέματος (I_{ag}):

Ο πράκτορας μπορεί να μεταφέρει υποσύνολα των 4 αντικειμένων {Brush, Key, Color, Code}, με περιορισμό χωρητικότητας ≤ 3 αντικείμενα.

- ο 0 αντικείμενα = 1 κατάσταση.
- ο 1 αντικείμενο = 4 καταστάσεις.
- ο 2 αντικείμενα = 6 καταστάσεις.
- ο 3 αντικείμενα = 4 καταστάσεις.
- ο Σύνολο καταστάσεων αποθέματος: $1 + 4 + 6 + 4 = 15$.

(Σημείωση: Στον κώδικα GridEnvironment.java, τα αντικείμενα έχουν σταθερές θέσεις στο grid (itemPositions HashMap). Όταν γίνεται pickup, το αντικείμενο προστίθεται στο inventory, αλλά η θέση του στο χάρτη παραμένει η ίδια. Έτσι, μετά

από drop, μπορεί να γίνει ξανά pickup από την ίδια θέση.) **Κατάσταση Στόχων (S_{targets}):**

Κάθε στόχος έχει δύο καταστάσεις: ολοκληρωμένος ή όχι.

- ο Τραπέζι: {Painted, Not_Painted}
- ο Καρέκλα: {Painted, Not_Painted}
- ο Πόρτα: {Open, Closed}
- ο **Σύνολο**: $2 * 2 * 2 = 8$ καταστάσεις.

3. Θέσεις Στόχων (P_{targets}):

Οι στόχοι (Τραπέζι, Καρέκλα, Πόρτα) τοποθετούνται σε έγκυρα κελιά και δεν μπορούν να αλληλοκαλύπτονται.

- ❖ Οι δυνατοί συνδυασμοί θέσεων είναι διατάξεις 3 στοιχείων σε 21 θέσεις: $P(21, 3) = 21 * 20 * 19 = 7.980$.

Συνολικό Μέγεθος Χώρου Καταστάσεων:

$$|S| = 21 * 15 * 8 * 7.980 = 20.109.600 \text{ καταστάσεις}$$

Το μέγεθος αυτό δικαιολογεί την ανάγκη χρήσης ευριστικών αλγορίθμων (όπως ο A*) αντί για τυφλή αναζήτηση, καθώς η πλήρης εξερεύνηση του δέντρου καταστάσεων θα ήταν εξαιρετικά δαπανηρή.

1.3. Ενέργειες και Μεταβάσεις

Ο πράκτορας διαθέτει το σύνολο ενεργειών $A = \{\text{move, pickup, drop, paint, open}\}$. Οι προϋποθέσεις και τα αποτελέσματα ορίζονται στην κλάση GridEnvironment.java:

- **move(direction):**

Pre: Η νέα θέση δεν είναι εκτός ορίων και δεν είναι εμπόδιο.

Eff: Η θέση του πράκτορα αλλάζει. Το κόστος αφαιρείται από τη χρησιμότητα.

Μετακίνηση σε {up, down, left, right}. Έγκυρη μόνο αν το νέο κελί είναι εντός ορίων και δεν είναι εμπόδιο.

- **pickup(item):**

Pre: Ο πράκτορας είναι στο ίδιο κελί με το αντικείμενο και $lag < 3$

Eff: Το αντικείμενο προστίθεται στο inventory.

Έγκυρη αν ο πράκτορας είναι στο ίδιο κελί με το αντικείμενο και έχει χώρο

στο inventory (<3).

- **drop(item):**

Pre: Ο πράκτορας κατέχει το αντικείμενο.

Eff: Το αντικείμενο αφαιρείται από το inventory.

- **paint(object):**

Pre: Ο πράκτορας είναι στη θέση του αντικειμένου (Table/Chair) και κατέχει {Brush, Color}.

Eff: Το αντικείμενο μαρκάρεται ως colored. Λαμβάνεται ανταμοιβή +1.0.

Αφαιρεί το αντικείμενο από το inventory.

Έγκυρη αν ο πράκτορας είναι στη θέση του αντικειμένου (Table/Chair) και κατέχει τα {Brush, Color}.

- **open():**

Pre: Ο πράκτορας είναι στη θέση της Door και κατέχει {Key, Code}.

Eff: Η πόρτα μαρκάρεται ως open. Λαμβάνεται ανταμοιβή +0.8

Έγκυρη αν ο πράκτορας είναι στη θέση της Door και κατέχει τα {Key, Code}.

1.4. Συνάρτηση Χρησιμότητας και Κόστους

Η συνάρτηση χρησιμότητας (Utility Function) που θέλουμε να μεγιστοποιήσουμε ορίζεται ως το άθροισμα των ανταμοιβών μείον το συσσωρευμένο κόστος.

Ανταμοιβές Στόχων (R_{goal}):

- $R(\text{Painted}(\text{Table})) = +1.0$
- $R(\text{Painted}(\text{Chair})) = +1.0$
- $R(\text{Open}(\text{Door})) = +0.8$
- **Μέγιστη δυνατή ανταμοιβή: 2.8**

Συνάρτηση Κόστους ($C(s, a)$):

Το κόστος κάθε ενέργειας μετακίνησης δεν είναι σταθερό, αλλά εξαρτάται από το φορτίο του πράκτορα. Ο κώδικας στο GridEnvironment.java (μέθοδος move) 11 ορίζεται το κόστος ανά βήμα ως:

$$\text{Cost}_{\text{step}} = C_{\text{base}} + (N_{\text{items}} * C_{\text{item}}) + (N_{\text{incompat}} * C_{\text{incompat}})$$

Όπου:

- $C_{base} = -0.01$: Βασικό κόστος ενέργειας.
- N_{items} : Αριθμός αντικειμένων που μεταφέρει.
- $C_{item} = -0.02$: Κόστος ανά μεταφερόμενο αντικείμενο.
- $N_{incompat}$: Αριθμός "ασύμβατων" αντικειμένων (αντικείμενα που δεν χρειάζονται πια για τους εναπομείναντες στόχους - π.χ. το brush/color είναι ασύμβατα μόνο αν table KAI chair είναι ήδη βαμμένα).
- $C_{incompat} = -0.03$: Πρόσθετη ποινή για ασύμβατα αντικείμενα.

Συνεπώς, η συνολική χρησιμότητα ενός επεισοδίου είναι:

$$U_{episode} = \Sigma(R_{goal}) - \Sigma_{t=1, \{Steps\}}(Cost_{step}(t))$$

Παρατήρηση:

Ο περιορισμός των 3 αντικειμένων δημιουργεί στον πράκτορα ένα πρόβλημα βελτιστοποίησης επιλογής. Ο πράκτορας πρέπει να επιλέγει έξυπνα ποια items να κρατήσει κάθε φορά, ώστε να ελαχιστοποιεί το κόστος μεταφοράς ασύμβατων αντικειμένων.

1.5. Μέτρο Απόδοσης (Performance Measure)

Το μέτρο απόδοσης που χρησιμοποιείται για την αξιολόγηση του πράκτορα ορίζεται στην κλάση GridEnvironment.java (μέθοδος getPerformance) ως:

$$Performance = U_{episode} / Steps$$

Αυτό το μέτρο ενθαρρύνει τον πράκτορα όχι μόνο να ολοκληρώσει τους στόχους (για να αυξήσει τον αριθμητή), αλλά να το κάνει με τον ελάχιστο δυνατό αριθμό βημάτων (για να μειώσει τον παρονομαστή), μεγιστοποιώντας την "πυκνότητα" της ανταμοιβής ανά χρονική στιγμή.

2. Υλοποίηση του πράκτορα

Η υλοποίηση του πράκτορα ακολουθεί την αρχιτεκτονική BDI (Belief-Desire-Intention) χρησιμοποιώντας το framework Jason για τη λογική του πράκτορα και Java για το περιβάλλον και τις βοηθητικές κλάσεις.

2.1. Αρχιτεκτονική Συστήματος

Το σύστημα αποτελείται από τρία κύρια συστατικά:

2.1.1. GridEnvironment.java

Υλοποιεί το περιβάλλον του πράκτορα και διαχειρίζεται:

- Το πλέγμα 5×5 με τα εμπόδια
- Τις θέσεις των αντικειμένων (σταθερές)
- Τις θέσεις των στόχων (δυναμικές)
- Την εκτέλεση των ενεργειών
- Τον υπολογισμό ανταμοιβών και κόστους
- Την παραγωγή αντιλήψεων (percepts) για τον πράκτορα

2.1.2. practical_reasoning_agent.asl

Υλοποιεί τη λογική του πράκτορα σε AgentSpeak(L):

- Πεποιθήσεις (beliefs) για την κατάσταση του κόσμου
- Επιθυμίες (desires) που εκφράζονται ως goals
- Προθέσεις (intentions) που υλοποιούνται ως plans

2.1.3. Βοηθητικές Κλάσεις

- **AStarPathfinder.java**: Αλγόριθμος A* για εύρεση βέλτιστης διαδρομής
- **ExperimentRunner.java**: Διαχείριση πειραμάτων και καταγραφή αποτελεσμάτων
- **calculate_goal_utilities.java**: Internal action για υπολογισμό χρησιμότητας στόχων
- **get_astar_path.java**: Internal action για κλήση A* από το ASL

2.2. Κύριος Βρόχος Ελέγχου

Ο κύριος βρόχος ελέγχου του πράκτορα ακολουθεί το παρακάτω σχήμα:

2.2.1. Ψευδοκώδικας

ΑΛΓΟΡΙΘΜΟΣ: Κύριος Βρόχος Ελέγχου

ΕΙΣΟΔΟΣ: Αρχική κατάσταση s_0

ΕΞΟΔΟΣ: Επίτευξη κατάστασης στόχου

1. Αρχικοποίηση πεποιθήσεων από αντιλήψεις
2. **ΟΣΟ** $\neg \text{goal_complete}()$ **KANE**
3. **ΑΝ** $\text{environment_changed}()$ **TOTE**
4. Επαναπρογραμματίσε τρέχουσα πλοήγηση
5. **ΤΕΛΟΣ_ΑΝ**
- 6.
7. $\text{στόχος} \leftarrow \text{ΕπιλογήΚαλύτερουΣτόχου}()$
- 8.
9. **ΑΝ** $\text{στόχος} = \text{paint_table}$ **Ή** $\text{στόχος} = \text{paint_chair}$ **TOTE**
10. **ΑΝ** $\neg \text{has_painting_items}()$ **TOTE**
11. Συλλογή Αντικειμένων Βαψίματος()
12. **ΑΛΛΙΩΣ**
13. Πλοήγηση Και Βάψιμο(στόχος)
14. **ΤΕΛΟΣ_ΑΝ**
15. **ΑΛΛΙΩΣ_ΑΝ** $\text{στόχος} = \text{open_door}$ **TOTE**
16. **ΑΝ** $\neg \text{has_door_items}()$ **TOTE**
17. Συλλογή Αντικειμένων Πόρτας()
18. **ΑΛΛΙΩΣ**
19. Πλοήγηση Και Άνοιγμα()
20. **ΤΕΛΟΣ_ΑΝ**
21. **ΤΕΛΟΣ_ΑΝ**
- 22.
23. Ενημέρωση Πεποιθήσεων()
24. **ΤΕΛΟΣ_ΟΣΟ**
25. Καταγραφή Αποτελεσμάτων()

2.2.2. Υλοποίηση σε AgentSpeak

Το αρχείο `practical_reasoning_agent.asl` υλοποιεί τον βρόχο ως εξής:

```
/* Εκκίνηση πράκτορα */
+!start : true
  <- .print("Agent started");
  !achieve_all_goals.
```

```
/* Κύριος βρόχος - Έλεγχος ολοκλήρωσης */  
+!achieve_all_goals : goal_complete(true)  
<- ?total_reward(R);  
  ?steps(S);  
  .print("=== SUCCESS! All goals achieved! ===");  
  .print("Utility: ", R, " | Steps: ", S);  
  !wait_for_reset.
```

```
/* Κύριος βρόχος - Συνέχεια εκτέλεσης */  
+!achieve_all_goals : true  
<- !select_and_execute_goal;  
  !achieve_all_goals. // ΑΝΑΔΡΟΜΗ
```

2.2.3. Ροή Εκτέλεσης

1. **Αρχικοποίηση:** Φόρτωση αρχικών πεποιθήσεων από το περιβάλλον
2. **Επιλογή στόχου:** Υπολογισμός βέλτιστου στόχου με βάση χρησιμότητα
3. **Σχεδιασμός:** Χρήση A* για εύρεση διαδρομής
4. **Εκτέλεση:** Ακολουθήση διαδρομής και εκτέλεση ενέργειας
5. **Επανάληψη:** Επιστροφή στο βήμα 2 μέχρι ολοκλήρωσης όλων των στόχων

2.3. Λογική Επιλογής Στόχου

Η επιλογή στόχου βασίζεται σε utility-based reasoning που μεγιστοποιεί τη χρησιμότητα ανά μονάδα απόστασης.

2.3.1. Αλγόριθμος Υπολογισμού Χρησιμότητας

ΣΥΝΑΡΤΗΣΗ: Υπολογισμός Χρησιμότητας Στόχου(στόχος, x, y)

1. βασική_ανταμοιβή ← ΛήψηΑνταμοιβήςΣτόχου(στόχος)
2. θέση_στόχου ← ΛήψηΘέσηςΣτόχου(στόχος)
3. απόσταση_στόχου ← AStarDistance(x, y, θέση_στόχου)
4. **ΑΝ** ΉχειΑπαραίτηταΑντικείμενα(στόχος) **TOTE**
5. κόστος_συλλογής ← ΕκτίμησηΚόστουςΣυλλογής(στόχος, x, y)
6. συνολική_απόσταση ← απόσταση_στόχου + κόστος_συλλογής
7. **ΑΛΛΙΩΣ**



8. συνολική_απόσταση $\leftarrow$ απόσταση_στόχου

9. **ΤΕΛΟΣ_ΑΝ**

10. χρησιμότητα $\leftarrow$ βασική_ανταμοιβή / συνολική_απόσταση

11. **ΕΠΙΣΤΡΟΦΗ** χρησιμότητα

2.3.2. Εκτίμηση Κόστους Συλλογής

Για να υπολογίσει το κόστος συλλογής απαραίτητων αντικειμένων, ο πράκτορας:

1. Εντοπίζει ποια items λείπουν για τον στόχο
2. Υπολογίζει την πραγματική απόσταση (με A^*) προς κάθε item
3. Προσθέτει τις αποστάσεις σειριακά (από τρέχουσα θέση $\rightarrow$ item1 $\rightarrow$ item2)

ΣΥΝΑΡΤΗΣΗ: ΕκτίμησηΚόστουςΣυλλογής(στόχος, fromX, fromY)

1. totalCost $\leftarrow$ 0

2. currentX $\leftarrow$ fromX

3. currentY $\leftarrow$ fromY

4. **ΓΙΑ_ΚΑΘΕ** item **ΣΤΟ** ΑπαραίτηταΑντικείμενα(στόχος) **ΚΑΝΕ**

5. **ΑΝ** $\neg$ ΈχειItem(item) **ΤΟΤΕ**

6. item_pos $\leftarrow$ ΛήψηΘέσηςItem(item)

7. pathLength $\leftarrow$ AStarDistance(currentX, currentY, item_pos)

8. totalCost $\leftarrow$ totalCost + pathLength

9. currentX $\leftarrow$ item_pos.x

10. currentY $\leftarrow$ item_pos.y

11. **ΤΕΛΟΣ_ΑΝ**

12. **ΤΕΛΟΣ_ΓΙΑ_ΚΑΘΕ**

13. **ΕΠΙΣΤΡΟΦΗ** totalCost

2.3.3. Αλγόριθμος Επιλογής Βέλτιστου Στόχου

ΑΛΓΟΡΙΘΜΟΣ: ΕπιλογήΚαλύτερουΣτόχου(x, y)

1. καλύτερος_στόχος $\leftarrow$ NULL

2. μέγιστη_χρησιμότητα $\leftarrow$ $-\infty$



```
3. ΓΙΑ_ΚΑΘΕ στόχος ΣΤΟ {paint_table, paint_chair, open_door} ΚΑΝΕ  
4. ΑΝ ¬ΈχειΟλοκληρωθεί(στόχος) ΤΟΤΕ  
5. χρησιμότητα ← ΥπολογισμόςΧρησιμότηταςΣτόχου(στόχος, x, y)  
6. .print("Utility(", στόχος, ") = ", χρησιμότητα)  
7.  
8. ΑΝ χρησιμότητα > μέγιστη_χρησιμότητα ΤΟΤΕ  
9. μέγιστη_χρησιμότητα ← χρησιμότητα  
10. καλύτερος_στόχος ← στόχος  
11. ΤΕΛΟΣ_ΑΝ  
12. ΤΕΛΟΣ_ΑΝ  
13. ΤΕΛΟΣ_ΓΙΑ_ΚΑΘΕ  
  
14. ΕΠΙΣΤΡΟΦΗ καλύτερος_στόχος
```

2.3.4. Υλοποίηση σε Java

Η κλάση calculate_goal_utilities.java υλοποιεί τον αλγόριθμο ως internal action:

@Override

```
public Object execute(TransitionSystem ts, Unifier un, Term[] args) {  
    int currentX = (int) ((NumberTerm) args[0]).solve();  
    int currentY = (int) ((NumberTerm) args[1]).solve();  
  
    boolean isTablePainted = hasBelief(ts, "colored", "table");  
    boolean isChairPainted = hasBelief(ts, "colored", "chair");  
    boolean hasBrush = hasBelief(ts, "carrying_item", "brush");  
    boolean hasColor = hasBelief(ts, "carrying_item", "color");  
  
    String bestGoal = "none";  
    double bestUtility = -1.0;  
  
    // Υπολογισμός utility για κάθε εκκρεμή στόχο  
    if (!isTablePainted) {  
        int[] tablePos = getPosition(ts, "table_at");  
        int collectionCost = estimateCollectionCost(ts, currentX, currentY,  
                                                    hasBrush, hasColor);  
        double utility = calculateGoalUtility(currentX, currentY,  
                                              tablePos[0], tablePos[1],  
                                              TABLE_REWARD, collectionCost);  
        if (utility > bestUtility) {  
            bestUtility = utility;  
            bestGoal = "table";  
        }  
    }  
}
```



```
// Παρόμοια για chair και door...
```

```
return un.unifies(args[2], new Atom(bestGoal));  
}
```

2.3.5. Γιατί Αυτή η Λογική Αυξάνει τη Χρησιμότητα

1. **Ελαχιστοποίηση Συνολικής Απόστασης:** Επιλέγοντας τον πλησιέστερο στόχο (με υψηλότερο utility/distance ratio), μειώνονται τα βήματα
2. **Προτεραιοποίηση Υψηλών Ανταμοιβών:** Οι στόχοι με reward=1.0 προτιμώνται έναντι του door (reward=0.8) όταν οι αποστάσεις είναι συγκρίσιμες
3. **Ενσωμάτωση Κόστους Συλλογής:** Λαμβάνεται υπόψη η επιπλέον απόσταση για συλλογή items, αποφεύγοντας "ψευδώς κοντινούς" στόχους
4. **Προσαρμοστικότητα:** Σε δυναμικό περιβάλλον, ο επανυπολογισμός επιτρέπει αλλαγή στόχου αν ένας στόχος απομακρυνθεί

2.4. Λογική Σχεδιασμού Δράσης

Ο σχεδιασμός δράσης χρησιμοποιεί τον αλγόριθμο A* με ευριστική Manhattan distance.

2.4.1. Αλγόριθμος A*

ΑΛΓΟΡΙΘΜΟΣ: AStarPathfinding(startX, startY, goalX, goalY)

1. openSet ← ΟυράΠροτεραιότητας()
2. closedSet ← Σύνολο()
3. startNode ← Node(startX, startY, g=0, h=Manhattan(start, goal))
4. openSet.add(startNode)

5. ΟΣΟ openSet ≠ ∅ **ΚΑΝΕ**

6. current ← openSet.poll() // Κόμβος με min f(n)

7.

8. ΑΝ current.position = goal **ΤΟΤΕ**

9. ΕΠΙΣΤΡΟΦΗ ReconstructPath(current)

10. ΤΕΛΟΣ_ΑΝ



```
11.
12. closedSet.add(current)
13.
14. ΓΙΑ_ΚΑΘΕ neighbor ΣΤΟ GetValidNeighbors(current) ΚΑΝΕ
15. ΑΝ neighbor ∈ closedSet ΤΟΤΕ ΣΥΝΕΧΙΣΕ
16.
17. tentative_g ← current.g + 1
18.
19. ΑΝ neighbor ∉ openSet Ή tentative_g < neighbor.g ΤΟΤΕ
20.   neighbor.g ← tentative_g
21.   neighbor.h ← Manhattan(neighbor, goal)
22.   neighbor.parent ← current
23.
24. ΑΝ neighbor ∉ openSet ΤΟΤΕ
25.   openSet.add(neighbor)
26. ΤΕΛΟΣ_ΑΝ
27. ΤΕΛΟΣ_ΑΝ
28. ΤΕΛΟΣ_ΓΙΑ_ΚΑΘΕ
29. ΤΕΛΟΣ_ΟΣΟ

30. ΕΠΙΣΤΡΟΦΗ [] // Δεν βρέθηκε διαδρομή
```

2.4.2. Ευριστική Συνάρτηση

Η ευριστική Manhattan distance ορίζεται ως:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Ιδιότητες:

- **Αποδεκτή (Admissible):** Ποτέ δεν υπερεκτιμά το πραγματικό κόστος
- **Συνεπής (Consistent):** $h(n) \leq c(n, n') + h(n')$ για κάθε διαδοχικό n'
- **Βέλτιστη για πλέγμα:** Είναι η ακριβής απόσταση χωρίς εμποδία

2.4.3. Υλοποίηση σε Java

```
public static List<String> findPath(int startX, int startY, int goalX, int goalY) {
    if (startX == goalX && startY == goalY) {
        return new ArrayList<>();
    }
}
```

```
PriorityQueue<Node> openSet = new PriorityQueue<>();
```



```
Set<Node> closedSet = new HashSet<>();

Node startNode = new Node(startX, startY, 0,
    manhattan(startX, startY, goalX, goalY), null);
openSet.add(startNode);

while (!openSet.isEmpty()) {
    Node current = openSet.poll();

    if (current.x == goalX && current.y == goalY) {
        return reconstructPath(current);
    }

    closedSet.add(current);

    // Εξερεύνηση γειτόνων (πάνω, κάτω, αριστερά, δεξιά)
    for (int[] dir : new int[][]{{1,0}, {-1,0}, {0,1}, {0,-1}}) {
        int newX = current.x + dir[0];
        int newY = current.y + dir[1];

        if (!isValidPosition(newX, newY)) continue;

        Node neighbor = new Node(newX, newY, current.g + 1,
            manhattan(newX, newY, goalX, goalY),
            current);

        if (closedSet.contains(neighbor)) continue;

        // Ενημέρωση ή προσθήκη στο openSet
        if (!openSet.contains(neighbor) || neighbor.g < existingG) {
            openSet.add(neighbor);
        }
    }
}

return new ArrayList<>(); // Δεν βρέθηκε path
}
```

2.4.4. Ολοκληρωμένη Διαδικασία Σχεδιασμού

ΑΛΓΟΡΙΘΜΟΣ: Εκτέλεση Σχεδίου Δράσης(targetX, targetY)

1. currentX, currentY $\leftarrow$ Λήψη Τρέχουσας Θέσης()
2. path $\leftarrow$ AStar(currentX, currentY, targetX, targetY)
3. **AN** path = [] **TOTE**
4. .print("ERROR: No path found")
5. Επιλογή Εναλλακτικού Στόχου()
6. **ΕΠΙΣΤΡΟΦΗ**
7. **ΤΕΛΟΣ_ΑΝ**
8. **ΓΙΑ_ΚΑΘΕ** direction **ΣΤΟ** path **ΚΑΝΕ**
9. move(direction)
10. Ενημέρωση Πεποιθήσεων()
- 11.
12. // Έλεγχος για δυναμικές αλλαγές
13. **AN** dynamicMode **ΚΑΙ** Στόχος Μετακινήθηκε() **TOTE**
14. .print("Target moved, re-planning...")
15. Εκτέλεση Σχεδίου Δράσης(Νέα Θέση Στόχου())
16. **ΕΠΙΣΤΡΟΦΗ**
17. **ΤΕΛΟΣ_ΑΝ**
18. **ΤΕΛΟΣ_ΓΙΑ_ΚΑΘΕ**
19. Εκτέλεση Τελικής Ενέργειας() // paint/open

2.4.5. Γιατί Αυτός ο Σχεδιασμός Αυξάνει τη Χρησιμότητα

1. **Βελτιστότητα Διαδρομής:** Ο A* εγγυάται τη συντομότερη διαδρομή, μειώνοντας το κόστος βημάτων
2. **Αποφυγή Εμποδίων:** Ο αλγόριθμος λαμβάνει υπόψη τα εμπόδια κατά την αναζήτηση
3. **Αποδοτικότητα:** Η ευριστική Manhattan μειώνει τον αριθμό κόμβων που εξερευνώνται
4. **Επαναπρογραμματισμός:** Σε δυναμικό περιβάλλον, η ανίχνευση αλλαγών και επανασχεδιασμός αποφεύγει άσκοπα βήματα

2.5. Διαχείριση Δυναμικότητας

Στο bonus σενάριο, το περιβάλλον αλλάζει κάθε 3 κινήσεις.

2.5.1. Μηχανισμός Ανίχνευσης Αλλαγών

Στο GridEnvironment.java:

```
if (dynamicMode && !isGoalComplete() && actionName.equals("move")) {  
    movementStepsSinceLastChange++;  
  
    if (movementStepsSinceLastChange >= STEPS_BETWEEN_CHANGES) {  
        randomizeGoalPositions(false);  
        movementStepsSinceLastChange = 0;  
        addPercept(Literal.parseLiteral("environment_changed"));  
    }  
}
```

2.5.2. Αντίδραση Πράκτορα

Στο practical_reasoning_agent.asl

```
/* Ανίχνευση αλλαγής περιβάλλοντος */  
+environment_changed : true  
  <- .print("WARNING: Environment changed!");  
  -environment_changed.  
  
/* Επαλήθευση θέσης στόχου πριν την τελική ενέργεια */  
+!paint_table : table_at(TX,TY) & carrying_item(brush) & carrying_item(color)  
  <- !navigate_to(TX,TY);  
    ?table_at(NewTX, NewTY);  
    ?position(MyX, MyY);  
    if (MyX == NewTX & MyY == NewTY) {  
        paint(table);  
    } else {  
        .print("Target moved, retrying...");  
        !paint_table;  
    }.  
}
```

2.5.3. Στρατηγική Προσαρμογής

1. **Επανελέγχος Θέσης:** Πριν από κάθε τελική ενέργεια, επιβεβαίωση ότι ο στόχος είναι ακόμα εκεί



2. **Επανασχεδιασμός:** Αν ο στόχος μετακινήθηκε, άμεσος επανυπολογισμός διαδρομής
3. **Επανεπιλογή Στόχου:** Αν ένας στόχος απομακρύνθηκε πολύ, επιλογή νέου στόχου με βάση updated utilities

2.6. Διαχείριση Αποθέματος (Inventory)

Ο πράκτορας διαχειρίζεται έξυπνα το inventory για να ελαχιστοποιεί το κόστος.

2.6.1. Στρατηγική "Smart Cleaning"

/* Απόρριψη περιττών αντικειμένων */

```
+!clean_inventory_smart : carrying_item(brush) & colored(table) & colored(chair)
<- .print("Dropping brush (no longer needed)");
drop(brush);
!clean_inventory_smart.
```

```
+!clean_inventory_smart : carrying_item(color) & colored(table) & colored(chair)
<- drop(color);
!clean_inventory_smart.
```

/* Απόρριψη door items αν χρειάζεται χώρος για painting */

```
+!clean_inventory_smart : carrying(3) & colored(table) & colored(chair) &
(carrying_item(brush) | carrying_item(color)) &
not open(door)
<- .print("Dropping painting items to make room for door items");
!drop_painting_items.
```

2.6.2. Λογική Απόρριψης

1. **Μετά από ολοκλήρωση στόχου:** Αν table KAI chair βαμμένα → drop brush/color
2. **Πριν από νέο στόχο:** Αν inventory γεμάτο και χρειάζονται νέα items → drop least important
3. **Αποφυγή ασύμβατων:** Προτεραιότητα στα items για τους εναπομείναντες στόχους

2.7. Σύνοψη Σχεδιαστικών Επιλογών

ΠΡΟΚΛΗΣΗ	ΛΥΣΗ	ΌΦΕΛΟΣ
ΕΠΙΛΟΓΗ ΣΤΟΧΟΥ	Utility-based με A* distance	Ελαχιστοποίηση συνολικής απόστασης
ΠΛΟΗΓΗΣΗ	A* με Manhattan heuristic	Βέλτιστες διαδρομές
INVENTORY	“Smart” cleaning based on goal status	Μείωση κόστους ασύμβατων items
ΔΥΝΑΜΙΚΟΤΗΤΑ	Re-planning και goal re-selection	Προσαρμογή σε αλλαγές
ΣΥΛΛΟΓΗ ITEMS	Εκτίμηση κόστους με πραγματικές αποστάσεις	Ρεαλιστικός προγραμματισμός

3. Πειραματική Ανάλυση

Ο πράκτορας δοκιμάστηκε σε 100 επεισόδια για κάθε σενάριο (στατικό και δυναμικό περιβάλλον), συλλέγοντας μετρικές για την αξιολόγηση της απόδοσής του.

3.1. Μεθοδολογία Πειραμάτων

3.1.1. Πειραματικές Ρυθμίσεις

- **Αριθμός Επεισοδίων:** 100 runs ανά σενάριο
- **Αρχική Θέση:** Πάντα (1,1)
- **Θέσεις Αντικειμένων:** Σταθερές (Brush:1,5 | Key:1,4 | Color:5,5 | Code:3,5)
- **Θέσεις Στόχων:** Τυχαιοποιούνται σε κάθε επεισόδιο
- **Στατικό Σενάριο:** Θέσεις στόχων σταθερές κατά τη διάρκεια επεισοδίου
- **Δυναμικό Σενάριο (Bonus):** Αλλαγή θέσεων στόχων κάθε 3 κινήσεις
- **Logging:** Αναλυτική καταγραφή κάθε ενέργειας και απόφασης

3.1.2. Μετρικές Αξιολόγησης

Για κάθε επεισόδιο καταγράφονται:

1. **Utility (U)**: Συνολική χρησιμότητα = $\Sigma(\text{Rewards}) - \Sigma(\text{Costs})$
2. **Steps**: Αριθμός κινήσεων (move actions)
3. **Total Actions**: Συνολικές ενέργειες (moves + pickups + drops + paint/open)
4. **Performance (P)**: $P = U / \text{Steps}$

3.1.3. Κλάση ExperimentRunner

Η κλάση ExperimentRunner.java διαχειρίζεται την εκτέλεση και καταγραφή:

```
public void recordRun(double utility, int movementSteps, int totalActions,  
    double performance, ...) {  
    allRuns.add(new RunStatistics(...));
```

```
// CSV logging για ανάλυση
```

```
simpleLogWriter.printf(Locale.US, "%d,%.4f,%d,%d,%.4f%n",  
    allRuns.size(), utility, movementSteps, totalActions, performance);
```

```
// Progress reporting κάθε 10 runs
```

```
if (allRuns.size() % 10 == 0) printProgressStatistics();
```

```
if (allRuns.size() >= maxRuns) finishExperiment();
```

```
}
```

3.2. Αποτελέσματα Πειραμάτων

3.2.1. Συγκριτικά Αποτελέσματα: Στατικό vs Δυναμικό

ΜΕΤΡΙΚΗ	ΣΤΑΤΙΚΟ	ΔΥΝΑΜΙΚΟ	ΔΙΑΦΟΡΑ
ΜΕΣΗ ΧΡΗΣΙΜΟΤΗΤΑ	1.8720	-0.0596	-103.2%
ΤΥΠΙΚΗ ΑΠΟΚΛΙΣΗ UTILITY	0.1393	1.4213	+920%
ΜΕΣΟΣ ΑΡΙΘΜΟΣ ΒΗΜΑΤΩΝ	28.32	67.84	+139.5%
ΤΥΠΙΚΗ ΑΠΟΚΛΙΣΗ STEPS	2.796	28.713	+927%
ΜΕΣΗ ΑΠΟΔΟΣΗ	0.0672	0.0082	-87.8%



ΤΥΠΙΚΗ ΑΠΟΚΛΙΣΗ PERFORMANCE	0.0116	0.0250	+116%
ΧΡΟΝΟΣ ΕΚΤΕΛΕΣΗΣ	3.05s	7.86s	+158%

3.2.2. Εύρος Τιμών

ΜΕΤΡΙΚΗ	ΣΤΑΤΙΚΟ [MIN, MAX]	ΔΥΝΑΜΙΚΟ [MIN, MAX]
UTILITY	[1.56, 2.24]	[-4.82, 2.23]
STEPS	[22, 34]	[23, 160]
ACTIONS	[31, 43]	[32, 169]
PERFORMANCE	[0.0459, 0.1018]	[-0.0301, 0.0970]

3.2.3. Ανάλυση Αποτελεσμάτων

Στατικό Περιβάλλον

Εξαιρετική Συνέπεια: Τυπική απόκλιση μόλις 0.14 (7.4% του μέσου)

Υψηλή Αξιοπιστία: Όλα τα runs έχουν θετική χρησιμότητα (1.56-2.24)

Προβλέψιμη Απόδοση: Εύρος βημάτων 22-34 (διακύμανση μόλις 12 βήματα)

Ταχύτητα Εκτέλεσης: Ολοκλήρωση σε 3.05s (100 runs)

Κατανομή Απόδοσης

ΚΑΤΗΓΟΡΙΑ	UTILITY RANGE	ΠΟΣΟΣΤΟ RUNS	ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ
ΑΡΙΣΤΑ	≥ 2.00	29%	Steps 22-26, optimal paths
ΠΟΛΥ ΚΑΛΑ	1.85-1.99	42%	Steps 27-29, καλή επιλογή στόχων
ΚΑΛΑ	1.70-1.84	24%	Steps 30-32, κάποιες suboptimal choices
ΜΕΤΡΙΑ	< 1.70	5%	Steps 33-34, δύσκολες διατάξεις

Συμπεράσματα Στατικού:

- **71% των runs** επιτυγχάνουν $utility \geq 1.85$
- Η στρατηγική utility-based goal selection λειτουργεί εξαιρετικά
- Ο A* εγγυάται βέλτιστες διαδρομές χωρίς re-planning overhead

*Δυναμικό Περιβάλλον***Προβλήματα που Παρατηρήθηκαν:**

Μέση χρησιμότητα **αρνητική** (-0.0596)

Τεράστια Διακύμανση: Τυπική απόκλιση 1.42 (utility από -4.82 έως +2.23)

Υπερβολικά Βήματα: Μέσος όρος 67.84 (vs 28.32 στατικό) - **+139.5%**

Ακραία Runs: 160 βήματα στο χειρότερο run (vs 34 στο στατικό)

Αρνητικές Χρησιμότητες: **45% των runs** έχουν αρνητική χρησιμότητα

Κατανομή Απόδοσης (Δυναμικό):

ΚΑΤΗΓΟΡΙΑ	UTILITY RANGE	ΠΟΣΟΣΤΟ RUNS	ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ
ΚΑΛΑ	≥ 1.50	11%	Steps < 40, ελάχιστες αλλαγές
ΜΕΤΡΙΑ	0.50-1.49	19%	Steps 40-60, κάποιες αλλαγές
ΧΑΜΗΛΑ	0.00-0.49	25%	Steps 60-80, πολλές αλλαγές
ΑΠΟΤΥΧΗΜΕΝΑ	< 0.00	45%	Steps > 80, συνεχείς αλλαγές

Αίτια Χαμηλής Απόδοσης στο Δυναμικό:

1. **"Target Chasing" Φαινόμενο:** Ο πράκτορας κυνηγάει στόχους που μετακινούνται συνεχώς
2. **Υπερβολικό Re-planning Overhead:** Κάθε 3 βήματα → νέα διαδρομή → wasted movements
3. **Καμία Πρόβλεψη:** Δεν υπάρχει μοντέλο πρόβλεψης μελλοντικών θέσεων

4. **Greedy Re-selection:** Η επιλογή νέου στόχου μπορεί να οδηγεί σε ακόμα χειρότερες επιλογές

Χειρότερες Περιπτώσεις:

RUN	UTILITY	STEPS	ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ
#78	-4.82	160	Ακραία περίπτωση: συνεχής μετακίνηση στόχων
#44	-3.68	138	Target chasing με πολλά re-plans
#51	-3.98	144	Αδυναμία ολοκλήρωσης λόγω συνεχών αλλαγών
#28	-3.61	139	Wasted movements προς μετακινημένους στόχους

Επιτυχημένες Περιπτώσεις στο Δυναμικό:

RUN	UTILITY	STEPS	ΕΠΕΞΗΓΗΣΗ
#98	2.17	25	Τύχη: ελάχιστες αλλαγές, καλές αρχικές θέσεις
#86	2.13	25	Γρήγορη ολοκλήρωση πριν πολλές αλλαγές
#24	2.23	23	Βέλτιστη διάταξη + γρήγορη εκτέλεση

3.2.4. Στατιστική Σύγκριση

Success Rate (Utility > 0):

- Στατικό: 100% (100/100 runs)
- Δυναμικό: 55% (55/100 runs)

Excellent Performance (Utility > 1.85):

- Στατικό: 71% (71/100 runs)
- Δυναμικό: 9% (9/100 runs)

Consistency (Coefficient of Variation):

- Στατικό: CV = 7.4% (εξαιρετική συνέπεια)
- Δυναμικό: CV = 2384% (εξαιρετικά υψηλή διακύμανση)



3.3. Ανάλυση Συμπεριφοράς Πράκτορα

3.3.1. Θεωρητικό Βέλτιστο vs Πραγματική Απόδοση

Υπολογισμός Θεωρητικού Βέλτιστου (Ιδανικό Σενάριο):

Ας υποθέσουμε βέλτιστες θέσεις στόχων με ελάχιστες αποστάσεις:

Αρχή (1,1) → Brush (1,5): 4 βήματα

Brush → Color (5,5): 4 βήματα

Color → Κοντινό Table (π.χ. 3,3): 2 βήματα [+1.0]

Table → Κοντινή Chair (π.χ. 2,4): 2 βήματα [+1.0]

Chair → Key (1,4): 1 βήμα

Key → Code (3,5): 2 βήματα

Code → Door (π.χ. 1,2): 3 βήματα [+0.8]

Σύνολο βημάτων: ~18-20 (minimum σε ιδανικό σενάριο)

Θεωρητικό Maximum Utility:

$$U_{\max} = 2.8 \text{ (rewards)} - \text{Steps}_{\min} \times \text{Cost}_{\text{per_step}}$$

Με βέλτιστη διαχείριση (2 items μέσος όρος, 0 incompatible):

$$\text{Cost}_{\text{per_step}} \approx 0.01 + 0.02 \times 2 = 0.05$$

$$U_{\max} \approx 2.8 - 18 \times 0.05 = 2.8 - 0.90 = 1.90$$

Με κάποιο overhead:

$$U_{\max} \approx 2.2 - 2.4$$

Επίτευξη Πράκτορα (Στατικό):

- **Μέση χρησιμότητα:** 1.872
- **Best run:** 2.24 (Run #63)
- **Ποσοστό επίτευξης:** $1.872 / 2.3 \approx 81.4\%$ του θεωρητικού βέλτιστου
- **Best case επίτευξη:** $2.24 / 2.4 \approx 93.3\%$ του θεωρητικού βέλτιστου

3.3.2. Ανάλυση Κόστους Μεταφοράς

Από τη συνάρτηση κόστους:

$$\text{Cost_step} = 0.01 + 0.02 \times \text{items} + 0.03 \times \text{incompatible}$$

Υπολογισμός Μέσου Κόστους (Στατικό):

$$\text{Από το: } U = 2.8 - \Sigma(\text{Cost_step})$$

$$1.872 \approx 2.8 - 28.32 \times \text{Cost_avg}$$

$$\text{Cost_avg} \approx 0.0328 \text{ per step}$$

Αναλύοντας:

$$0.0328 = 0.01 + 0.02 \times \text{items_avg} + 0.03 \times \text{incompatible_avg}$$

$$\text{Αν items_avg} \approx 1.0-1.2:$$

$$0.0328 \approx 0.01 + 0.02 \times 1.1 + 0.03 \times \text{incompatible}$$

$$\text{incompatible} \approx 0.06 \text{ (εξαιρετικά χαμηλό!)}$$

Συμπέρασμα: Ο πράκτορας διατηρεί ελάχιστα ασύμβατα items (~0.06 κατά μέσο όρο), υποδεικνύοντας:

Αποτελεσματική στρατηγική `clean_inventory_smart`

Έγκαιρη απόρριψη περιττών αντικειμένων

Προτεραιοποίηση συμβατών items για εναπομείναντες στόχους

3.3.3. Απόδοση Πλοήγησης (A*)

Μετρικές Στατικού Περιβάλλοντος:

1. Failure Rate:

- **0%** σε όλα τα runs (100/100 επιτυχείς navigations)
- Ο A* βρίσκει πάντα διαδρομή

2. Consistency:

- Τυπική απόκλιση βημάτων: 2.796
- Προβλέψιμη συμπεριφορά ανεξάρτητα από διάταξη

Μετρικές Δυναμικού Περιβάλλοντος:

1. Re-planning Frequency:

- Κάθε 3 βήματα → αλλαγή περιβάλλοντος
- Μέσος όρος 67.84 βήματα → ~22.6 re-plans ανά run

2. Wasted Movement:

- Διαφορά βημάτων: $67.84 - 28.32 = 39.52$ **wasted steps**
- **58%** των βημάτων είναι "σπαταλημένα" λόγω αλλαγών

3. Failure to Converge:

- 45% runs έχουν αρνητική χρησιμότητα
- Αδυναμία ολοκλήρωσης στόχων πριν εξαντληθεί η χρησιμότητα

3.4. Επίδραση Δυναμικότητας

3.4.1. Ποσοτική Σύγκριση

ΜΕΤΡΙΚΗ	ΣΤΑΤΙΚΟ	ΔΥΝΑΜΙΚΟ	ΜΕΤΑΒΟΛΗ
ΜΕΣΗ UTILITY	1.872	-0.060	-103.2%
ΜΕΣΑ STEPS	28.32	67.84	+139.5%
ΜΕΣΗ PERFORMANCE	0.0672	0.0082	-87.8%
ΤΥΠΙΚΗ ΑΠΟΚΛΙΣΗ UTILITY	0.139	1.421	+920%
SUCCESS RATE (U>0)	100%	55%	-45%

3.4.2. Αίτια Καταστροφικής Απόδοσης

1. Μηχανισμός "Target Chasing":

Παράδειγμα Run #78 (Utility: -4.82, Steps: 160):

Βήμα 0: Door at (3,3) - distance 4

Βήμα 3: Door moved to (5,5) - distance 6 (from new position)

Βήμα 6: Door moved to (1,5) - distance 8

Βήμα 9: Door moved to (4,2) - distance 7



...

Βήμα 150+: Πράκτορας ακόμα κυνηγάει

Κάθε αλλαγή → Re-planning → Wasted 3 βήματα

Συνολικά: $\sim 50 \text{ re-plans} \times 3 = 150 \text{ wasted movements}$

2. Έλλειψη Πρόβλεψης:

Ο πράκτορας δεν έχει μοντέλο για να προβλέψει:

- Που θα μετακινηθεί ένας στόχος;
- Ποιος στόχος είναι πιθανότερο να μείνει σταθερός;
- Πότε να εγκαταλείψει έναν στόχο;

3.4.3. Επιτυχημένα Runs στο Δυναμικό

Χαρακτηριστικά Runs με Utility > 1.50:

Γρήγορη Εκτέλεση: Ολοκλήρωση σε < 35 βήματα

Τυχερές Διατάξεις: Αρχικές θέσεις στόχων κοντά στον πράκτορα

Σταθεροί Στόχοι: Οι τυχαίες αλλαγές δεν απομάκρυναν πολύ τους στόχους

Βέλτιστη Σειρά: Utility-based selection έτυχε να επιλέξει βέλτιστη σειρά

Παράδειγμα: Run #24 (Best in Dynamic):

Utility: 2.23 | Steps: 23 | Performance: 0.0970

Ανάλυση:

- Γρήγορη συλλογή Brush+Color (12 βήματα)
- Table κοντά
- Chair κοντά (2 βήματα) - Painted πριν αλλαγή
- Γρήγορη συλλογή Key+Code (8 βήματα)
- Door άνοιξε στο βήμα 23

3.5. Περιπτώσεις Αποτυχίας και Όρια

3.5.1. Παρατηρημένες Αποτυχίες

Στατικό Περιβάλλον:

- 0 runs με αρνητική χρησιμότητα
- 0 navigation failures
- 0 timeouts ή infinite loops

Δυναμικό Περιβάλλον:

- **45 runs με αρνητική χρησιμότητα** (45%)
- **0 navigation failures** (ο A* πάντα βρίσκει path)
- **Απώλεια χρησιμότητας** λόγω συσσώρευσης κόστους

3.5.2. Ανάλυση Αποτυχημένων Runs

Κατηγοριοποίηση Runs με $U < 0$ (Dynamic):

UTILITY RANGE	ΑΡΙΘΜΟΣ RUNS	ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ
-0.5 ΕΩΣ 0.0	15	Μέτρια προβλήματα, 65-80 βήματα
-1.0 ΕΩΣ -0.5	11	Σοβαρά προβλήματα, 80-100 βήματα
-2.0 ΕΩΣ -1.0	10	Πολύ σοβαρά, 100-120 βήματα
< -2.0	9	Καταστροφικά, 120+ βήματα

Κοινά Μοτίβα σε Αποτυχημένα Runs:

1. **Πολλαπλά Re-plans:** > 20 αλλαγές περιβάλλοντος
2. **Μεγάλες Μετατοπίσεις:** Στόχοι μετακινούνται > 5 κελιά κάθε φορά
3. **Άτυχες Timing:** Πράκτορας φτάνει ακριβώς πριν την αλλαγή
4. **Zigzag Pattern:** Διαδοχικές αλλαγές σε αντίθετες κατευθύνσεις

3.5.3. Θεωρητικοί Περιορισμοί

1. Worst-Case Scenario στο Δυναμικό:

Θεωρητικά χειρότερο:

- Κάθε 3 βήματα: στόχος μετακινείται στη μέγιστη απόσταση
- Μέγιστη απόσταση στο 5×5 grid: 8 (Manhattan)
- Ατέρμων βρόχος: Ο πράκτορας δεν φτάνει ποτέ

Στην πράξη:

Παρατηρήθηκε max 160 βήματα (Run #78)

Μετά από ~160 βήματα, η χρησιμότητα έχει καταρρεύσει

2. Greedy Selection Limitations:

Ο utility-based greedy selection είναι **τοπικά βέλτιστος** όχι globally.

Παράδειγμα υποβέλτιστης σειράς:

Αρχική κατάσταση:

- Table: 10 βήματα μακριά (utility = $1.0/10 = 0.10$)
- Door: 2 βήματα μακριά + 8 για items (utility = $0.8/10 = 0.08$)

Greedy επιλέγει: Table ($0.10 > 0.08$)

Αλλά αν Door ήταν πρώτο:

- Μετά Door → Table: συνολικά $10+2 = 12$ βήματα
- Μετά Table → Door: συνολικά $10+8 = 18$ βήματα

Διαφορά: $6 \text{ βήματα} \times 0.05 = 0.30$ utility loss

3.5.4. Όρια Υλοποίησης

Τεχνικά Όρια:

Πτυχή	Όριο	Επίπτωση
Grid Size	5×5	Η πολυπλοκότητα A^* είναι $O(n^2)$, κλιμακώσιμο μέχρι ~20×20
Inventory	3 items	Απαιτεί έξυπνη διαχείριση



Re-planning	Κάθε action	Overhead στο δυναμικό
Lookahead	0 (greedy)	Τοπικά βέλτιστες επιλογές

Αρχιτεκτονικά Όρια:

Απουσία Μνήμης: Κάθε επιλογή στόχου είναι stateless

Απουσία Μάθησης: Δεν μαθαίνει από προηγούμενες αλλαγές

Απουσία Πρόβλεψης: Δεν μοντελοποιεί μελλοντικές καταστάσεις